

Exercise 7:

```
syms f(x,y,z);
zlow=-1;zup=3;ylow=0;yup=6;xlow=-4;xup=4;
f(x,y,z)= z*(x^3-2*x*y-z);
analytical = double(int(int(int(f,z,zlow,zup),y,ylow,yup),x,xlow,xup));
built_in = integral3(@integrand,xlow,xup,ylow,yup,zlow,zup);
Trapezoidal = d3_trapezoidal(@integrand,xlow,xup,ylow,yup,zlow,zup,20);
Simpson = simpson_3d(@integrand,xlow,xup,ylow,yup,zlow,zup,20);
fprintf("Analytical = %f \n" + ...
    "Buil in integral3 = %f \n" + ...
    "Trapezoidal = %f \n" + ...
    "Simpson = %f \n",analytical,built_in,Trapezoidal,Simpson)
```

```
Analytical = -448.000000
Buil in integral3 = -448.000000
Trapezoidal = -449.280000
Simpson = -448.000000
```

```
abs((analytical-Trapezoidal)/analytical)*100
```

```
ans = 0.2857
```

Comparison: We can see that the Analytical result, the built in integral3 function result, and the Simpson numerical result are the same. The Trapezoidal numerical result is the only result that is different with a percentage error of 0.29%.

```
function [m] = integrand(x,y,z)
m= z.*(x.^3-2*x.*y-z);
end

function [I] = d3_trapezoidal(f,ax,bx,ay,by,az,bz,n)
I = 0;
h=(bz-az)/n;% for the y limits
for i = 0:n
    z = az + i*h;
    if (i==0 || i==n)
        I = I + d2_trapezoidal(f,z,ax,bx,ay,by,n);
    else
        I = I + 2*d2_trapezoidal(f,z,ax,bx,ay,by,n);
    end
end
I = I*h/2;
end

function [I] = d2_trapezoidal(f,z,ax,bx,ay,by,n)
I = 0;
h=(by-ay)/n;% for the y limits
for i = 0:n
    y = ay + i*h;
    if (i==0 || i==n)
        I = I + segment_trapezoidal(f,z,y,ax,bx,n);
    else
        I = I + 2*segment_trapezoidal(f,z,y,ax,bx,n);
    end
end
```

```

        I = I + 2*segment_trapezoidal(f,z,y,ax,bx,n);
    end
end
I = I*h/2;
end

function [I] = segment_trapezoidal(f,z,y,a,b,n)
I=0; % initial value of integral
h=(b-a)/n;
for i=0:n
    x= a + i*h; % goin from a to b
    if (i==0 || i==n)
        I = I + f(x,y,z);
    else % if is not 0 or n
        I = I + 2*f(x,y,z);
    end
end
I = I*h/2;
end

% Simpson:
function [I] = simpson_3d(f,ax,bx,ay,by,az,bz,n) % "calculate" inner int
if (mod(n,2)==0)
    I=0;
    h = (bz-az)/n;
    for i =0:n
        z = az + i*h;
        if (i==0 || i==n)
            I = I + simpson_one_third2d(f,z,ax,bx,ay,by,n);
        elseif (mod(i,2)==1)
            I = I + 4*simpson_one_third2d(f,z,ax,bx,ay,by,n);
        else
            I = I + 2*simpson_one_third2d(f,z,ax,bx,ay,by,n);
        end
    end
    I = I*h/3;
else
    disp("n has to be even")
end
end

function [I] = simpson_one_third2d(f,z,ax,bx,ay,by,n) % "calculate" inner int
if (mod(n,2)==0)
    I=0;
    h = (by-ay)/n;
    for i =0:n
        y = ay + i*h;
        if (i==0 || i==n)
            I = I + simpson_one_third(f,z,y,ax,bx,n);
        elseif (mod(i,2)==1)
            I = I + 4*simpson_one_third(f,z,y,ax,bx,n);
        else
            I = I + 2*simpson_one_third(f,z,y,ax,bx,n);
        end
    end
end

```

```

    end
    I = I*h/3;
else
    disp("n has to be even")
end
end

function [I] = simpson_one_third(f,z,y,ax,bx,n) % this calculates outer integral
if (mod(n,2)==0)
    I=0;
    h = (bx-ax)/n;
    for i =0:n
        x = ax + i*h;
        if (i==0 || i==n)
            I = I + f(x,y,z);
        elseif (mod(i,2)==1)
            I = I + 4*f(x,y,z);
        else
            I = I + 2*f(x,y,z);
        end
    end
    I = I*h/3;
else
    disp("n has to be even")
end
end

```